Activity 3: Build an Accessible Razor Pages Shell

Course: Build ASP.NET Web Apps with Vibe Coding (TGS-2023040477)

Alignment: LO2 · K2/A2

Duration: 55 minutes

Runnable snapshot: src/LocalCRM

Goal

Create semantic navigation, visible keyboard focus and a labelled customer-contact entry point.

What you will produce

an accessible layout and Contacts index page with a reproducible keyboard checklist

Safety and data boundary

Use only the supplied fictional records in mock-data/. Do not paste personal, customer, credential, or production data into an AI tool. Review every generated diff before running it.

Vibe script

> Update the Razor layout and add Pages/Contacts/Index.cshtml. Use native header/nav/main/footer landmarks, one h1, descriptive link text and visible :focus-visible styles. No clickable divs, positive tabindex or ARIA when native HTML suffices.

Step-by-step

Step 1 — Run the baseline

Start LocalCRM and inspect the current DOM and accessibility tree in browser developer tools.

```
dotnet run --project LocalCRM --urls http://localhost:5050
```

Step 2 — Submit the accessibility prompt

Ask for a patch limited to _Layout.cshtml, site.css and the new Contacts page; review the proposed landmarks.

Step 3 — Create the Contacts page

Add a page with one h1 and descriptive navigation.

Create or update LocalCRM/Pages/Contacts/Index.cshtml:

```
@page
@model LocalCRM.Pages.Contacts.IndexModel
@{ ViewData["Title"] = "CRM contacts"; }
<h1>CRM contacts</h1>
<p>Review fictional training contacts.</p>
<a class="btn btn-primary" asp-page="./Create">Add a contact</a>
```

Step 4 — Add its PageModel

Keep the initial GET handler intentionally empty.

Create or update LocalCRM/Pages/Contacts/Index.cshtml.cs:

```
using Microsoft.AspNetCore.Mvc.RazorPages;
namespace LocalCRM.Pages.Contacts;
```

```
public sealed class IndexModel : PageModel
{
    public void OnGet() { }
}
```

Step 5 — Restore focus visibility

Add an explicit projector-visible keyboard focus token.

Create or update LocalCRM/wwwroot/css/site.css excerpt:

```
a:focus-visible, button:focus-visible, input:focus-visible, select:focus-visible {
  outline: 3px solid #1f6feb;
  outline-offset: 3px;
}
```

Step 6 — Build and navigate

Compile, open /Contacts, and verify the generated link target.

```
dotnet build LocalCRM/LocalCRM.csproj
curl -i http://localhost:5050/Contacts
```

Step 7 — Run the keyboard checklist

Starting at the address bar, use Tab and Shift+Tab only. Record focus order and confirm Enter opens Create.

Expected evidence: All interactive elements receive visible focus in DOM order.

Step 8 — Inspect the accessibility tree

Confirm header/navigation/main/footer landmarks and a single level-one heading; save evidence/activity-03.md.

Run the supplied snapshot

The Activity folder already contains a complete stage-appropriate ASP.NET Core project and mock data. From this Activity folder:

```
cd src/LocalCRM
dotnet restore
dotnet run --urls http://localhost:5103
```

Open http://localhost:5103. For Activities 7–10, also probe http://localhost:5103/api/contacts; for Activities 8–10, probe /openapi/v1.json and /health/ready.

Acceptance checklist

□ The project targets net10.0 and restores without an unreviewed package.

□ The app starts from this Activity folder without relying on another Activity.

□ The supplied mock contacts are fictional and load as described.

□ Positive and negative paths match the expected status or UI state.

□ The diff contains only intended files and no secret.

Troubleshooting

- NETSDK1045: install a .NET 10 SDK or select the installed 10.0 feature band in global.json.
- Address already in use: stop the previous Activity process or use --urls http://localhost:PORT with a free port.
- HTTP 404: compare the exact route, method, route constraint and Activity stage.

- HTTP 400/415: inspect ProblemDetails and confirm JSON shape plus Content-Type: application/json.
- SQLite failure (Activities 8–10): remove only the local training localcrm.db, then restart so the snapshot recreates mock data.

Reflection and evidence record

1. Explain how the evidence demonstrates LO2 and K2/A2.

2. Name one AI suggestion you rejected or changed and why.

3. Identify one negative path you tested and the observable result.

4. State the next improvement you would make without widening the approved scope.

Evidence item	Record the path, command result or observation
Build or test result	
Runtime URL and HTTP status	
Positive-path evidence	
Negative-path evidence	
Reviewed AI change or rejection	
Commit or evidence reference	